Unidad II · Actividad 6

Control de acceso según el rol

Fundamentos de Arquitectura de Software y Desarrollo Backend

Tema 2. Control de acceso de usuarios 2.1 Gestión de roles por usuario · 2.2 Validación de permisos por rol

Actividad #9. Implementación y prueba del control de acceso a rutas según el rol del usuario.

El proyecto ya sabe qué puede hacer cada rol. La actividad 5 todavía deja que el cliente escriba `actorCompanyUserId`. Esta semana ese id sale de un token que entrega `login`. La ruta deja de creer el identificador que viene en la operación.

Se sigue en el Sistema General de Ingresos: Python, Strawberry, GraphQL, SQLAlchemy y MySQL. No se abre otro proyecto y no se vuelve a diseñar la matriz de permisos.

1. El cliente ya no elige quién es

Hasta ahora GraphiQL podía decir “soy Ana” poniendo su membresía en la operación. El servidor llamaba `exigir_permiso` con ese número. Servía para probar la lista del rol. No comprobaba la identidad.

El control de acceso cambia el origen de ese número. La persona entra con correo y contraseña. Si el hash coincide, el servidor firma un token y lo devuelve. Las siguientes peticiones lo envían en el encabezado HTTP. El resolver lee la membresía desde el token, busca el rol activo y aplica el permiso que esa ruta ya exigía.

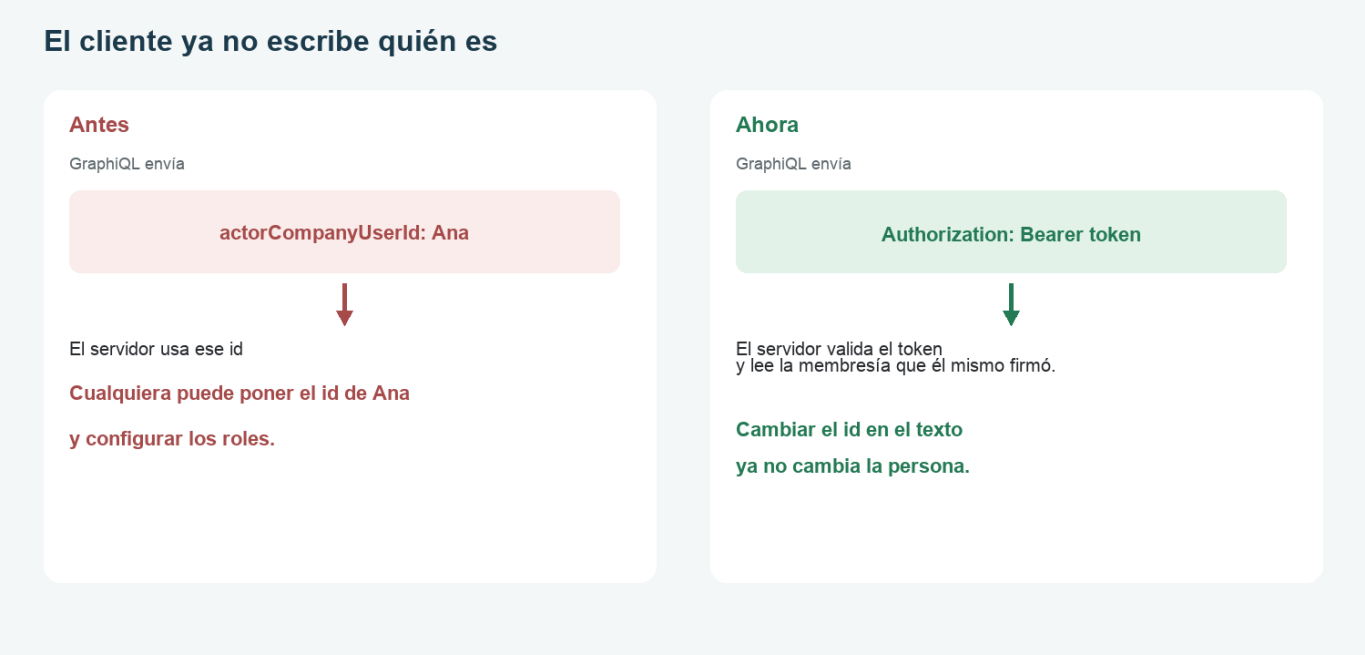


Figura 1. El id de la membresía deja de ser un argumento de la operación.

En este sistema, una **ruta** es una operación GraphQL que el cliente puede llamar: `createAccount`, `createTransaction`, `setRolePermissions`, `accounts`. No hace falta crear una API REST nueva. La puerta es la operación. El rol decide si esa puerta se abre.

2. El token: generación y validación

JWT significa **JSON Web Token**.

- **JSON** es el texto de los datos, entre llaves: la membresía, la empresa de esa membresía y la fecha de vencimiento.
- **Web** quiere decir que ese texto viaja en la petición HTTP, en el encabezado `Authorization`.
- **Token** es la credencial de esa entrada. El servidor la firma. No es la contraseña y no sustituye el hash guardado en `users`.

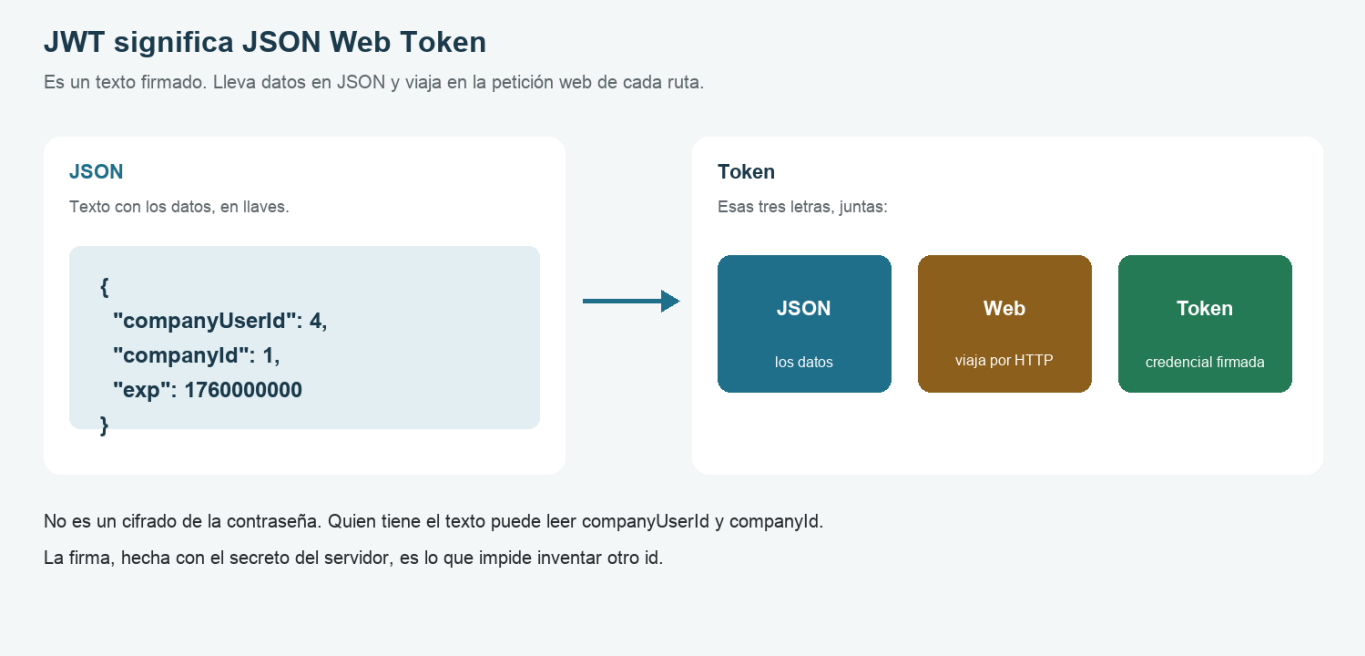


Figura 2. JWT es un texto firmado. Quien lo ve puede leer los datos. No puede cambiarlos sin el secreto.

Ese texto va en tres partes separadas por un punto.



Figura 3. Los datos se pueden leer. Sin el secreto no se puede firmar otro `companyUserId` *ni otro* `companyId`.

Los datos que van en el token son pocos:

Dato	Para qué
`companyUserId`	La membresía con la que entró la persona
`companyId`	La empresa de esa membresía
`exp`	Momento en que el token deja de servir

No pongas la contraseña, el hash ni la lista de permisos. La lista puede cambiar mientras el token sigue vigente. Si viajara dentro del token, el capturista conservaría los permisos viejos hasta que `exp` venciera.

Cómo se genera en `login`

`login` ya compara el hash. Si la contraseña es correcta, además firma el token. La herramienta de este laboratorio es **PyJWT**.

```
import jwt
from datetime import datetime, timedelta, timezone

def crear_token(company_user_id: int, company_id: int) -> str:
    ahora = datetime.now(timezone.utc)
    datos = {
        "companyUserId": company_user_id,
        "companyId": company_id,
        "exp": ahora + timedelta(minutes=30),
    }
    return jwt.encode(datos, SECRETO, algorithm="HS256")
```

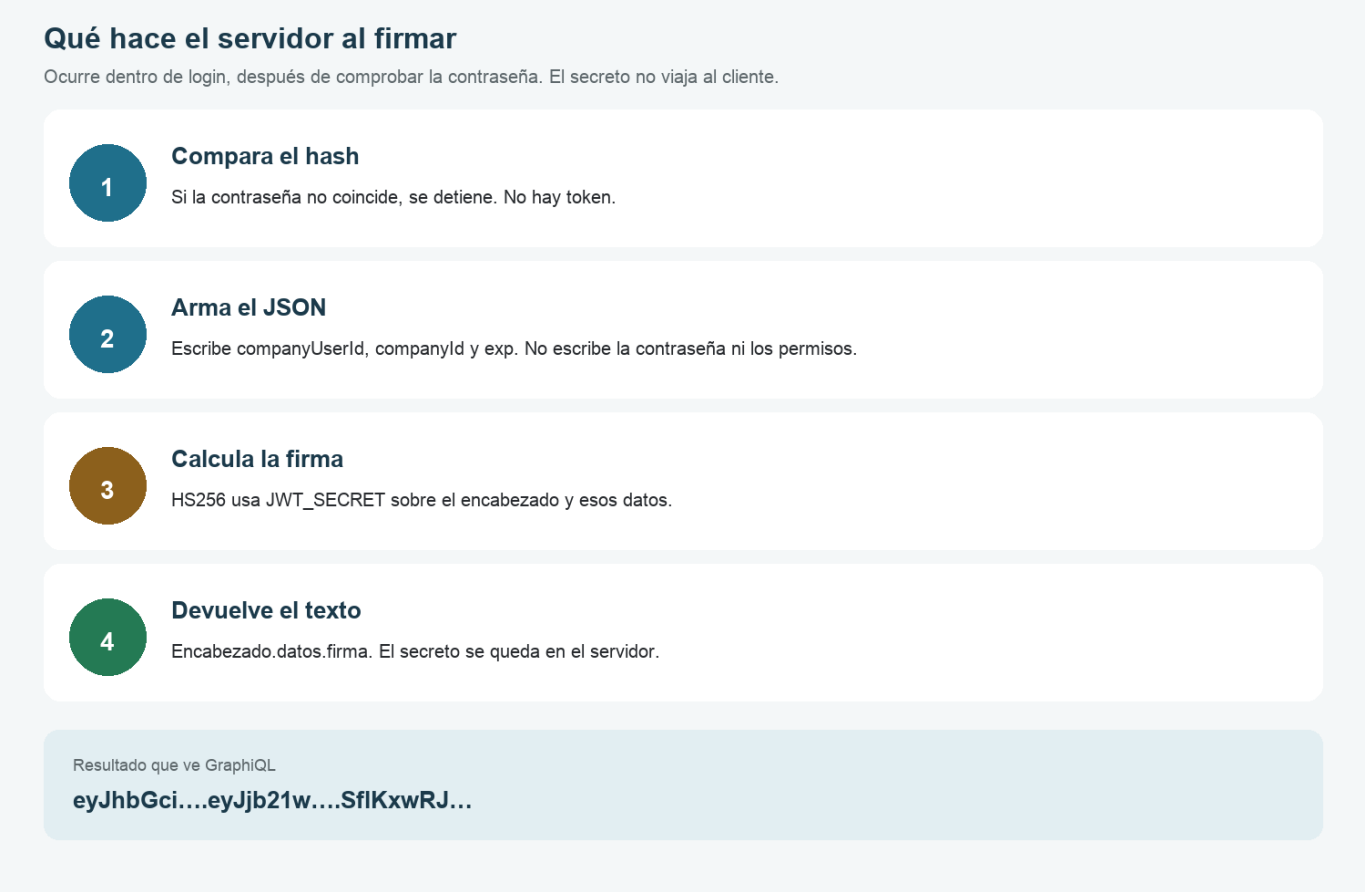


Figura 4. Firmar es trabajo del servidor. GraphiQL solo recibe el texto ya armado.

La firma usa un secreto, no un par de llaves

Firmar aquí no cifra el token. Calcula una marca sobre el encabezado y los datos. Si alguien cambia `companyUserId` o `companyId`, esa marca ya no coincide.

`HS256` es criptografía de **un solo secreto**. La misma cadena firma y comprueba. No hay llave pública ni llave privada. El algoritmo con dos llaves se llama, por ejemplo, `RS256`: la privada firma y la pública solo comprueba. Este laboratorio no lo usa.

El secreto **no se comparte con el cliente**. GraphiQL recibe el token. No recibe `JWT_SECRET`. Si el estudiante tuviera el secreto, podría firmar un token de Ana.

Para crearlo, una sola vez, en la máquina del servidor:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Ese texto de 64 caracteres se guarda en el ambiente, con el nombre `JWT_SECRET`. No va en el código, ni en Git, ni en la respuesta de `login`. El proceso de la API lo lee al firmar y al validar. Si más adelante hay dos servidores, los dos usan el mismo secreto. El navegador no.

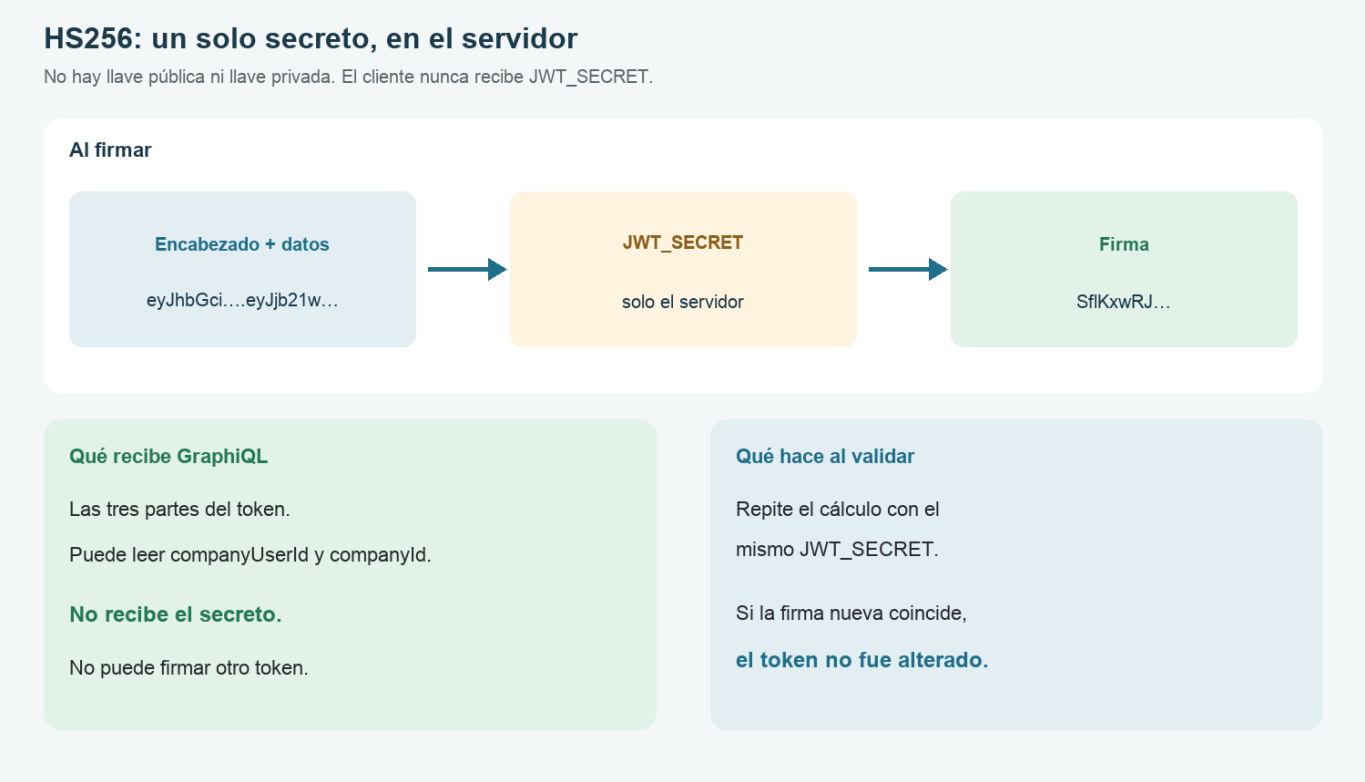


Figura 5. El cliente presenta el token. Solo el servidor, con el mismo secreto, puede decir si la firma es auténtica.

`HS256` es el único algoritmo que se acepta al decodificar. Así un token que diga “sin algoritmo” no pasa.

`companyId` lo copia el servidor desde la fila de esa membresía. El cliente no lo elige. Si la persona pertenece a dos empresas, entra dos veces y obtiene dos tokens. Cada token vale para una sola empresa.

La respuesta sigue siendo `AuthPayload`: `token`, `tokenType: "bearer"` y los datos mínimos del usuario. El tipo `User` sigue sin exponer el hash.

Cómo se valida en cada petición

El cliente repite el token en el encabezado:

```
Authorization: Bearer eyJhbGciOi...
```

Cada ruta protegida hace tres preguntas, en este orden:

1. ¿Vino el encabezado y la firma es válida?
2. ¿`exp` todavía no pasó?
3. ¿La membresía existe y su empresa es el `companyId` del token?
4. ¿El rol activo de esa membresía tiene el permiso de esta ruta?

```
def leer_actor(db, info) -> int:
    encabezado = info.context["request"].headers.get("authorization", "")
    if not encabezado.startswith("Bearer "):
        raise PermissionError("Falta el token")
    try:
        datos = jwt.decode(
            encabezado.removeprefix("Bearer ").strip(),
            SECRETO,
            algorithms=["HS256"],
        )
```

```
    except jwt.PyJWTError:
        raise PermissionError("El token no es válido")

    company_user_id = int(datos["companyUserId"])
    membresia = db.get(CompanyUserModel, company_user_id)
    if membresia is None or not membresia.is_active:
        raise PermissionError("La membresía no existe o no está activa")
    if membresia.company_id != int(datos["companyId"]):
        raise PermissionError("El token no corresponde a esa empresa")
    return company_user_id
```

`jwt.PyJWTError` cubre una firma alterada y un token vencido. No hace falta un mensaje distinto para cada caso en esta entrega. Ninguno de los dos guarda la fila.

La operación GraphQL no trae otro id de empresa. Cambiar de compañía exige otro `login` y otro token.

`GraphQLRouter`, de `strawberry.fastapi`, ya pone la petición HTTP en el contexto. No hace falta otro `context_getter` si el proyecto no reemplazó ese contexto. El encabezado llega así al resolver, GraphQL ejecuta la operación y `exigir_permiso` sigue decidiendo con el rol.

```
from strawberry.fastapi import GraphQLRouter

router = GraphQLRouter(schema)
```

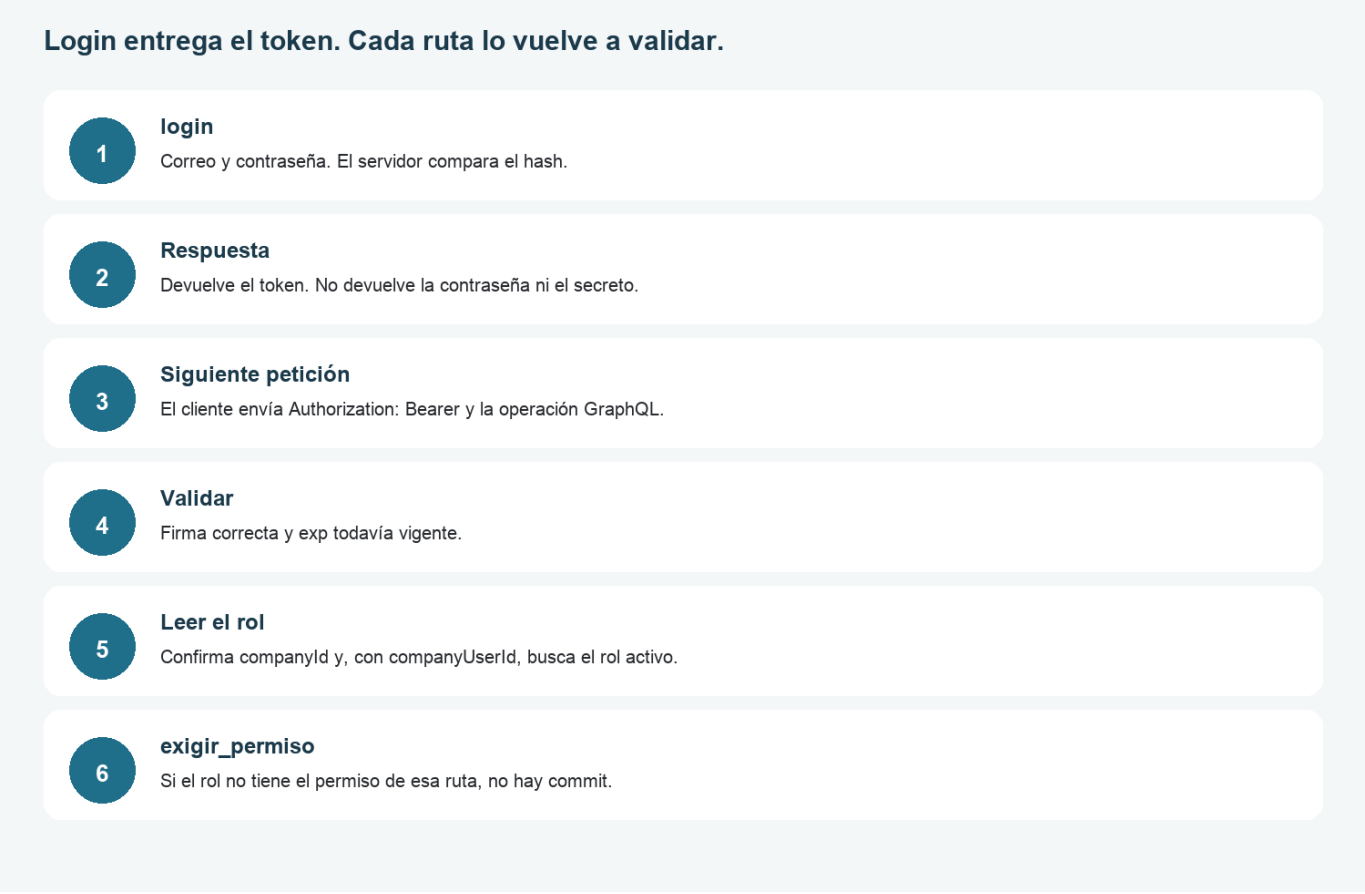


Figura 6. El token se crea una vez. Cada ruta lo valida de nuevo y vuelve a leer el rol.

Qué herramientas se combinan

Pieza	De dónde viene	Qué hace aquí
Hash de la contraseña	Actividades anteriores	`login` comprueba la credencial
PyJWT	Biblioteca nueva	Firma y valida el token con `HS256`
`Authorization: Bearer`	Encabezado HTTP	Lleva el token sin ponerlo en la operación GraphQL
`GraphQLRouter`	`strawberry.fastapi`	Pasa la petición al resolver
`company_user_roles`	Actividad 3	Dice el rol activo de esa membresía
`role_permissions` y `exigir_permiso`	Actividades 3 a 5	Dicen si ese rol puede usar la ruta
Variable de ambiente	`JWT_SECRET`	Guarda el secreto fuera del código

Otra forma de token es un texto aleatorio guardado en MySQL: el servidor lo busca en cada petición y puede borrarlo para cerrar la sesión antes de tiempo. Esta actividad no usa esa tabla. Usa JWT, con una vigencia corta, porque la firma se comprueba sin una consulta extra y el rol igual se lee en la base.

3. El rol del usuario se lee en cada petición

El puesto de la persona no se copia al token. `assignRole` sigue siendo la operación que cambia el rol de una membresía, y sigue exigiendo `roles.write`. Quien no es administrador no asigna puestos.

Después de `assignRole`, la siguiente petición del mismo token ya ve el rol nuevo. El token no cambió: cambió la fila activa en `company_user_roles`. Por eso Pedro puede entrar como consulta y, cuando Ana lo pasa a capturista, la misma sesión puede registrar un movimiento.

Una membresía sigue con un solo rol activo. El token no agrega un segundo puesto.

4. La misma ruta responde distinto según el rol

`createAccount` es la misma operación para María y para Luis. El permiso sigue siendo `accounts.write`. María es capturista y no lo tiene. Luis es contador y sí. Los dos presentan un token válido. La diferencia es el rol que la base devuelve para cada membresía.



Figura 7. Un token válido no abre todas las rutas. Abre las que el rol tiene concedidas.

Estas son las rutas que dejan de recibir `actorCompanyUserId`. El permiso no cambia.

Ruta	Permiso
`createCompanyUser`, `deactivateCompanyUser`	`company.users.write`
`accounts`, `createAccount`	`accounts.read`, `accounts.write`
`concepts`, `createConcept`, `assignConceptToAccount`	`concepts.read`, `concepts.write`
`transactions`, `createTransaction`	`transactions.read`, `transactions.write`
`permissionCatalog`, `previewRolePermissions`	`roles.read`
`setRolePermissions`, `assignRole`, `assignPermissionToRole`	`roles.write`

`login` no pide token: es la ruta que lo crea. `createCompany` y el primer `createCompanyAdmin` tampoco, porque todavía no hay una sesión de esa empresa.

La empresa del token y la empresa de la cuenta tienen que ser la misma. Un token de otra empresa no usa las cuentas de esta.

5. Brechas si el token se usa mal

Un JWT válido no significa que la persona frente a la pantalla sea la dueña del token. Estas son las fallas que esta entrega debe evitar. No agregan otra funcionalidad: cambian cómo se guarda el secreto y qué se pone dentro del token.

Brecha	Qué pasa	Qué se hace en este laboratorio
Token copiado	Quien lee el encabezado puede usarlo hasta `exp`. El servidor ve una firma correcta.	Vigencia de 30 minutos. No pegar el token completo en capturas, chats ni GitHub.
Datos legibles	La firma no esconde `companyUserId` ni `companyId`. El texto no está cifrado.	No poner la contraseña, el hash ni la lista de permisos en el token.
Secreto filtrado	Si el secreto está en el código, otra persona firma un token de Ana y el servidor lo acepta.	`JWT_SECRET` en el ambiente. Al validar, solo se acepta `HS256`.
Token alterado o vencido	Cambian el id, o `exp` ya pasó.	`jwt.decode` rechaza la petición y no se escribe.



Figura 8. Copiar un token ajeno o publicar el secreto deja el control de acceso en manos de otra persona.

Segunda parte. Actividad para entregar

Continúa el proyecto de las actividades 1 a 5. Instala PyJWT en el ambiente que ya usan. El secreto queda en el ambiente, con un ejemplo sin valor real si el equipo documenta la variable.

No se pide limitar los intentos de `login` ni armar otra lista de permisos. No se pide un token guardado en MySQL.

Parte 1. `login` firma un token

Cuando el correo y la contraseña son correctos, `login` devuelve un JWT de 30 minutos con `companyUserId`, `companyId` y `exp`, firmado con `HS256`. `companyId` es el de la membresía, no uno enviado por el cliente.

- El payload no contiene la contraseña ni los permisos.
- Un correo o contraseña incorrectos no entregan token.
- El secreto no aparece en el código fuente ni en la respuesta.

En GraphiQL el estudiante hace dos pasos, los dos contra `/graphql`.

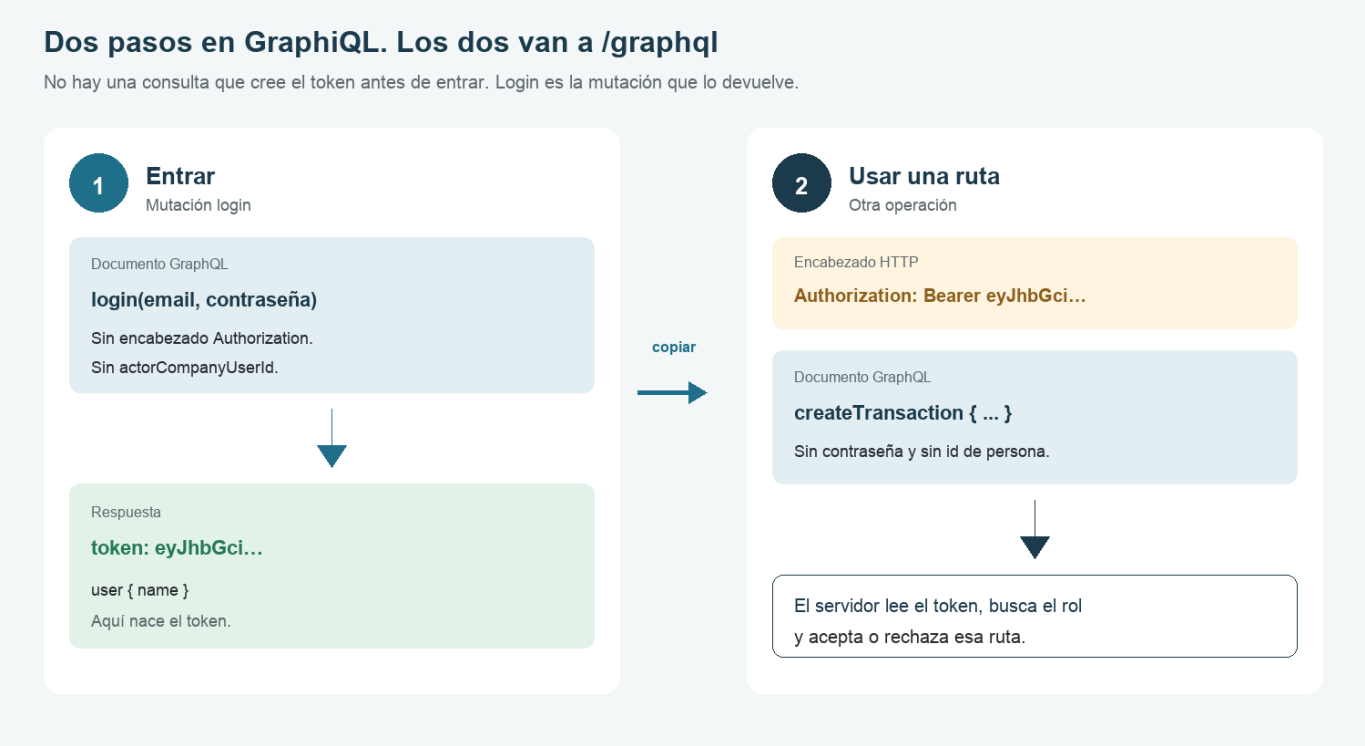


Figura 9. El token nace en la respuesta de `login`. La ruta siguiente solo lo reenvía en el encabezado.

Primero ejecuta la mutación `login`. No hay una consulta previa que cree el token. Esta respuesta es la que lo trae:

```
mutation {
  login(input: { email: "maria@empresa.test", password: "secreto" }) {
    tokenType
    token
    user {
      name
    }
  }
}
```

Si el correo o la contraseña fallan, no viene `token`. Si entran, copia ese texto.

Después abre los encabezados HTTP de GraphiQL y escribe:

```
Authorization: Bearer TOKEN_QUE_DEVOLVIO_LOGIN
```

A partir de ahí lanza las otras operaciones, por ejemplo `createAccount` o `createTransaction`, sin `actorCompanyUserId` y sin volver a mandar la contraseña. El token va en el encabezado, no dentro del documento GraphQL.

Si esa pantalla no muestra encabezados, el mismo par de peticiones se hace con un cliente HTTP contra `/graphql`. La evidencia tiene que mostrar el encabezado y el resultado, no el secreto.

Parte 2. La ruta toma la persona del token

Quita `actorCompanyUserId` de las operaciones de la tabla anterior. El resolver llama `leer_actor` y después `exigir_permiso`, con el mismo permiso de antes.

- Sin encabezado, la operación responde `errors` y no guarda.

- Un token con una letra cambiada responde `errors` y no guarda.
- Un token vencido responde `errors` y no guarda.
- Mandar el id de otra persona dentro de la operación ya no cambia quién llama, porque ese argumento ya no existe.

Parte 3. Pruebas con los mismos puestos

Usa la empresa y las personas de las actividades 4 y 5. Cada persona entra con `login` y usa su propio token.

Caso	Token	Ruta	Resultado
1	Ninguno	`createAccount`	Se rechaza y no hay fila nueva
2	María, capturista	`createTransaction`	Se guarda, si la cuenta y el concepto ya son válidos
3	María, capturista	`createAccount`	Se rechaza y no hay fila nueva
4	Luis, contador	`createAccount`	Se guarda
5	Pedro, consulta	`createTransaction`	Se rechaza y no hay fila nueva
6	Ana	`assignRole` de Pedro a `CAPTURISTA`	Se guarda
7	El mismo token de Pedro	`createTransaction`	Ahora se guarda, porque el rol se leyó otra vez en MySQL
8	Token de otra empresa	`accounts` de esta empresa	Se rechaza
9	Token alterado o vencido	`createAccount`	Se rechaza y no hay fila nueva

El caso 7 usa el token que Pedro obtuvo antes del caso 6. No hace falta un `login` nuevo para ver el rol nuevo.

Entrega

- El proyecto actualizado, con `login`, la lectura del token y las rutas de la tabla sin `actorCompanyUserId`.
- Un ejemplo de variable de ambiente, sin el secreto real.
- Captura de un `login` correcto y de uno rechazado. Del token correcto, muestra solo los datos legibles (`companyUserId`, `companyId` y `exp`), no la firma completa ni el secreto.
- Captura de los casos 1 a 9, con el encabezado visible y una frase: qué regla demuestra.
- En los rechazos de escritura, una consulta que muestre que la tabla no ganó esa fila.

No incluyas el ambiente virtual, el archivo del secreto ni tokens vigentes en el repositorio.

Criterios

- El token lo firma el servidor después de comprobar el hash.
- La firma usa `HS256` y un secreto que no está en el código.
- El token lleva la membresía, la empresa de esa membresía y la vigencia. No lleva la contraseña ni la lista de permisos.
- Cada ruta protegida obtiene la persona del token y sigue usando `exigir_permiso`.
- Sin token, con token alterado o con token vencido, no se escribe.
- María y Luis llaman la misma ruta y el resultado depende del rol.
- Cambiar el rol de Pedro se nota en la siguiente petición, con el token anterior.
- Un token de otra empresa no lee las cuentas de esta.
- `login` sigue sin devolver el hash.